PYTHON PROGRAMMING LAB MANUAL

Week 4 — Functions and Modules

Program	Mechanical Engineering	**Semester**	IV
Course Name	Programming Concepts in Mechanical Engineering	**Course Code**	25ME34I
Week	4	**Duration**	3 Hours (Practice Session)
Course Outcome	CO-1: Use appropriate syntaxes while writing program.	**Program Outcomes**	PO-1, PO-2, PO-4

Learning Objectives

- Define functions with parameters and return values, and call them to solve engineering problems.
- Understand the difference between local and global variable scope.
- Write short lambda (anonymous) functions for simple one-line tasks.
- Distinguish built-in functions from user-defined functions.
- Import and use standard Python modules such as math.

Quick Theory Recap

- Function definition: def function_name(parameters): … — a reusable block of code that can accept inputs (parameters) and give back a result using return.
- Scope: a variable created inside a function is local to that function; a variable created outside any function is global and can be read anywhere.
- Lambda function: a short, unnamed function written as lambda arguments: expression, useful for small one-line operations.
- Built-in functions (e.g. print(), len(), max()) are ready-made and come with Python; user-defined functions are written by the programmer for a specific task.
- Modules: import brings in extra functionality, e.g. import math gives access to functions like sqrt(), sin(), cos(), and constants like math.pi.

Practical Exercises

Exercise 1: Area of a Circle Using a Function

Aim: Define a function that calculates and returns the area of a circle for a given radius.

Concept:

- Area of a circle = $\pi \times r^2$

Program:

```
# Program to calculate the area of a circle using a function

def area_of_circle(radius):
    pi = 3.14159
    area = pi * radius * radius
    return area

# Input from user
r = float(input("Enter the radius of the circle: "))

# Function call
result = area_of_circle(r)
print("Area of the circle is:", result)
```

Sample Output

```
Enter the radius of the circle: 7
Area of the circle is: 153.93791
```

Explanation: area_of_circle(radius) is a user-defined function with one parameter. It computes the area and sends the value back to the caller using return, where it is stored in result and printed.

Exercise 2: Greatest Common Divisor (GCD) Using a Function

Aim: Write a function that finds the GCD of two numbers entered by the user.

Concept:

- GCD (Greatest Common Divisor) is the largest number that divides both given numbers exactly.
- Euclid's method: repeatedly replace the larger number with the remainder of dividing it by the smaller number, until the remainder is 0.

Program:

```
# Program to find the GCD of two numbers using a function

def find_gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Input from user
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

# Function call
gcd = find_gcd(num1, num2)
print("GCD of", num1, "and", num2, "is:", gcd)
```

Sample Output

```
Enter first number: 36
Enter second number: 24
GCD of 36 and 24 is: 12
```

Explanation: find_gcd(a, b) repeatedly applies a, b = b, a % b (Euclid's algorithm) until b becomes 0; at that point a holds the GCD, which is returned to the caller.

Exercise 3: Lambda Functions — Squaring a Number

Aim: Explore lambda functions by writing a small one-liner that squares a number.

Program:

```
# Program to demonstrate a lambda function

# A lambda function to square a number
square = lambda x: x * x

# Input from user
n = float(input("Enter a number: "))

# Using the lambda function
print("Square of", n, "is:", square(n))
```

Sample Output

```
Enter a number: 6
Square of 6.0 is: 36.0
```

*Explanation: lambda x: x * x defines a small, unnamed function in a single line and assigns it to square. It works exactly like a normal function defined with def, but is best suited for short, simple tasks.*

Note: *A few more one-liner lambda examples for practice: cube = lambda x: x ** 3, is_even = lambda x: x % 2 == 0, add = lambda a, b: a + b.*

Exercise 4: Using the math Module for Trigonometric Calculations

Aim: Use Python's built-in math module to perform trigonometric calculations.

Program:

```
# Program to perform trigonometric calculations using the math module

import math

# Input angle in degrees
angle_deg = float(input("Enter angle in degrees: "))

# Convert degrees to radians (math functions expect radians)
angle_rad = math.radians(angle_deg)
```

```
# Trigonometric calculations
sin_val = math.sin(angle_rad)
cos_val = math.cos(angle_rad)
tan_val = math.tan(angle_rad)

# Display results
print("sin(", angle_deg, ") =", round(sin_val, 4))
print("cos(", angle_deg, ") =", round(cos_val, 4))
print("tan(", angle_deg, ") =", round(tan_val, 4))
```

Sample Output

```
Enter angle in degrees: 30
sin( 30.0 ) = 0.5
cos( 30.0 ) = 0.866
tan( 30.0 ) = 0.5774
```

Explanation: import math gives access to ready-made trigonometric functions. Since math.sin(), math.cos(), and math.tan() expect the angle in radians, math.radians() is used first to convert the user's degree input.

Self-Practice (To Be Completed Individually)

Attempt the following on your own and get your output verified by the faculty:

- Write a function volume_of_cylinder(radius, height) that returns the volume of a cylinder ($V = \pi \times r^2 \times h$).
- Write a function to find the Least Common Multiple (LCM) of two numbers (hint: LCM = (a × b) / GCD(a, b)).
- Write a lambda function that converts a temperature from Celsius to Fahrenheit.
- Write a program using the math module to calculate the hypotenuse of a right triangle given the two other sides (math.sqrt and math.pow, or math.hypot).
- Write a function that takes a list of numbers and returns the largest value, without using the built-in max() function.

Reference: Week 4 syllabus content (25ME34I — Programming Concepts in Mechanical Engineering, DTE Karnataka).